Algebra and Discrete Mathematics
ADM_A_B

doc. Xiaolu Hou, PhD.

FIIT, STU
xiaolu.hou @ stuba.sk

Course Outline

- Vectors and matrices
- System of linear equations
- Matrix inverse and determinants
- Vector spaces, bases and matrix transformations
- Quiz one
- Matrix transformations, fundamental spaces and decompositions
- Eulerian tours and Hamiltonian cycles
- Paths and spanning trees
- Quiz two
- Trees and networks
- Matching

Recommended reading

- Saoub, K. R. (2017). A tour through graph theory. Chapman and Hall/CRC.
  - Sections 4.3, 4.4, 7.4, 7.5
  - Accessible online (free copy)
  - Alternative download link

Lecture outline

- Shortest networks
- Steiner Trees
- Metric Traveling Salesman Problem
- Flow and capacity
- Rooted trees

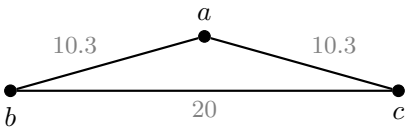
Trees and networks

- Shortest networks
- Steiner Trees
- Metric Traveling Salesman Problem
- Flow and capacity
- Rooted trees

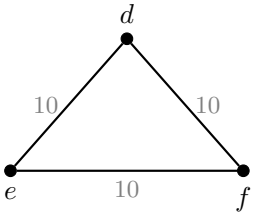
Shortest network

- Network: a connected graph
- A shortest network is a connected network of least total weight.

MST



T_1



T_2

- Edge lengths are drawn to scale since the angle created by the edges will play a large role in determining where to place shortcuts
- The minimum spanning tree consists of the two edges of length 10.3 in T_1, and any two edges in T_2.
- Could we do better than a minimum spanning tree?

Fermat point

- A similar question was posed by the 17th century French mathematician Pierre de Fermat in a letter to Evangelista Torricelli, an Italian physicist and mathematician
- In his letter, Fermat challenged Torricelli to find a point that minimizes the sum of the distances to the three vertices of a triangle.

Definition

A *Fermat point* of a triangle is a point p for which the sum of the distances from p to the three vertices is minimized. When all angles of the triangle are less than $120°$, the three angles formed at p are each $120°$.

Shortest network for a triangle

Theorem

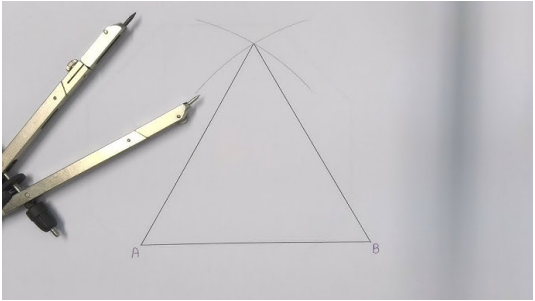
Given three points and the triangle T determined by them, the shortest network connecting the three points is either

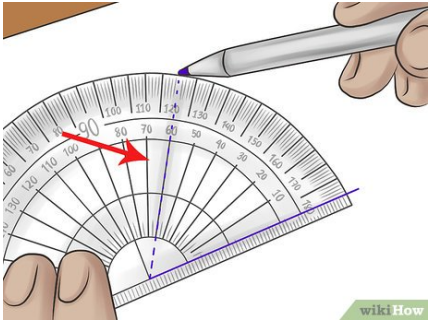
- *the two shortest sides of T provided T has one angle of at least 120°*
- *the three segments connecting the Fermat point for T to the original three vertices of T*

Torricelli's Construction

- Input: Triangle $T = \triangle abc$ where all angles measure less than $120°$
- Steps
  1. Along edge ab of T construct an equilateral triangle using that edge and a new point x that is on the opposite side of the edge as c
  2. Repeat step 1 for the other two edges of T, introducing new points y and z across from b and a, respectively
  3. Join x and c, y and b, and z and a by a line segment
  4. The common intersection point of these three line segments is the Fermat point p of T.
  5. The shortest network is the line segments joining each of the original vertices, a, b, and c, with p
- Output: Fermat point p and shortest network connecting a, b, and c

Torricelli's Construction – tools



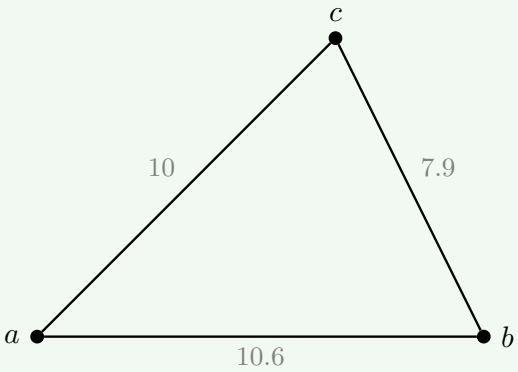


Note

Can be done using a ruler and a compass (or a protractor)

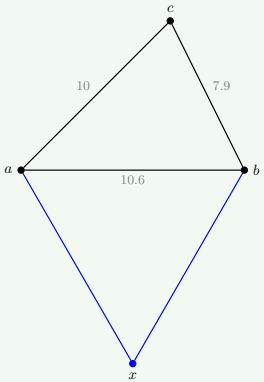
Torricelli's Construction – example

Example



Torricelli's Construction – example

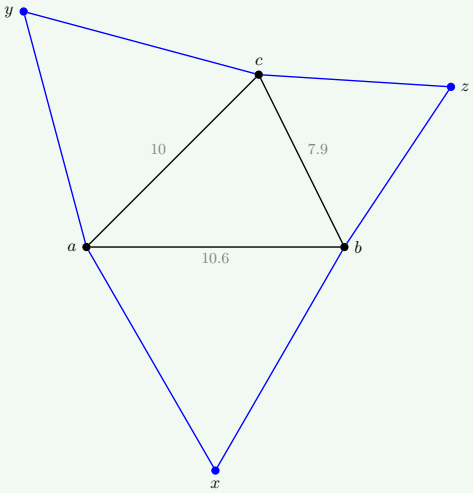
Example



- Step 1. Form an equilateral triangle off edge ab with new point x on the opposite side of ab as c

Torricelli's Construction – example

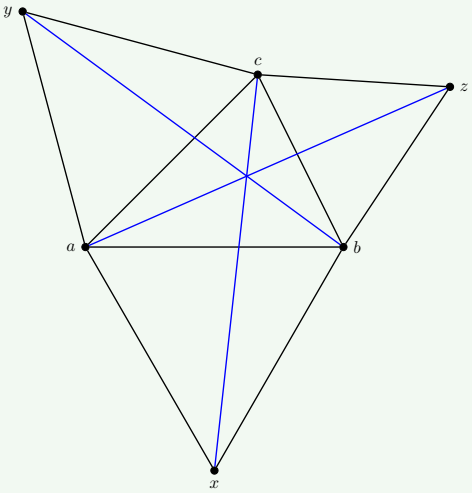
Example



- Step 2. Repeat the construction for edges ac and bc.

Torricelli's Construction – example

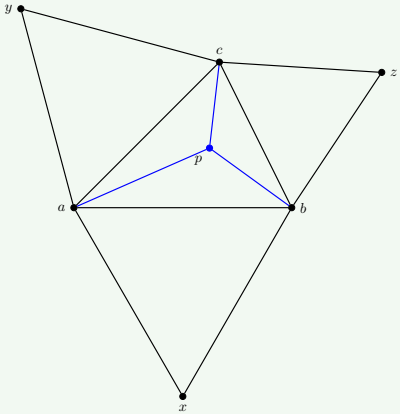
Example



- Step 3. Join x and c, y and b, and z and a

Torricelli's Construction – example

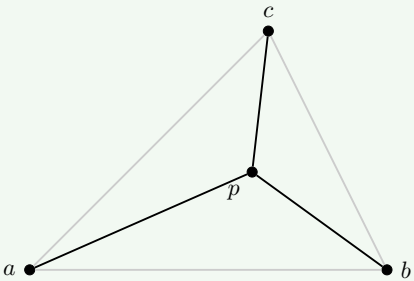
Example



- Step 4. Find the Fermat point p as the common intersection point of the three constructed segments.
- Step 5. Highlight the edges from p to each of the original vertices

Torricelli's Construction – example

Example



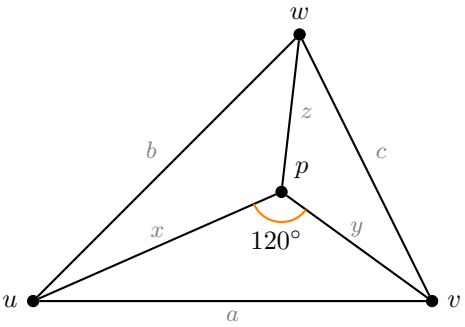
- Output: the shortest network consists of the three segments joining p to the original vertices.

Torricelli's Construction – length of the shortest network

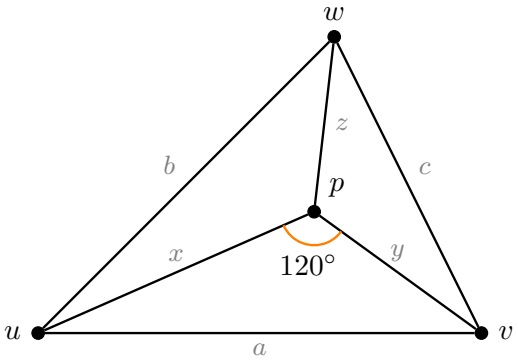
- Law of cosines

$$
\begin{aligned}
a^2 &= b^2 + c^2 - 2bc\cos(\angle uwv) \\
c^2 &= a^2 + b^2 - 2ab\cos(\angle wuv) \\
b^2 &= a^2 + c^2 - 2ac\cos(\angle wvu)
\end{aligned}
$$

- Compute the angles of the triangle
- Assign coordinates to vertices
- Solve for the lengths x, y, and z



Torricelli's Construction – length of the shortest network



- If ℓ denotes the length of the shortest network connecting vertices of a triangle having angles that measure less than $120°$ each and sides of lengths a, b, c

$$\ell = x + y + z = \sqrt{\frac{a^2+b^2+c^2}{2} + \frac{\sqrt{3}}{2}\sqrt{2a^2b^2+2a^2c^2+2b^2c^2-(a^4+b^4+c^4)}}$$

Torricelli's Construction – length of the shortest network

Example

- Continuing from the previous example

$$a = 7.9, \quad b = 10, \quad c = 10.6$$

- With the formula we get the length of the network obtained

$$\ell = \sqrt{\frac{274.77}{2} + \frac{\sqrt{3}}{2}\sqrt{48978.7752 - 26519.7777}} \approx 16.345$$

- In comparison, the MST has a total length of 17.9
- The shortest network saves 1.555, or 8.69%

Trees and networks

- Shortest networks
- Steiner Trees
- Metric Traveling Salesman Problem
- Flow and capacity
- Rooted trees

Steiner Trees

Definition

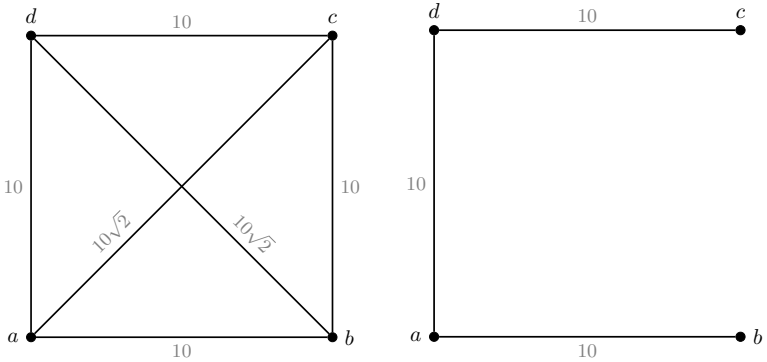
- For a graph G, a *Steiner point* is a new point p added to the graph with degree 3, such that the three edges incident to p meet at angles of $120°$.
- A *Steiner tree* is a tree whose vertices consist of the original vertices of G together with possible Steiner points.

- A Steiner point is similar to a Fermat point in that it is added to a graph to help find a shortest network connecting the original vertices.
- It can be shown that finding a shortest network amounts to finding a minimum Steiner tree.
- This problem is named for the 19th century Swiss mathematician Jakob Steiner

Finding Steiner Trees

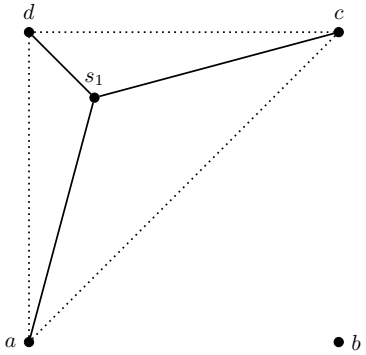
- Although the Steiner Tree Problem sounds fairly simple, it belongs to a class of problems known as NP-hard.
  - proposed solutions can be verified quickly
  - finding an optimal solution can be quite difficult

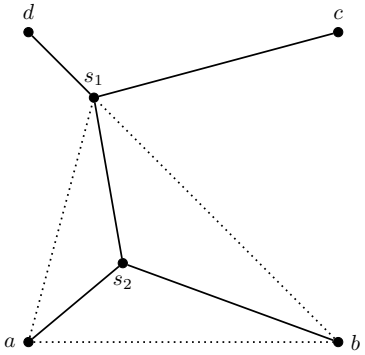
Four vertices – MST



- A graph with four vertices that lie on a square
- A minimum spanning tree consists of three of the four edges of length 10.

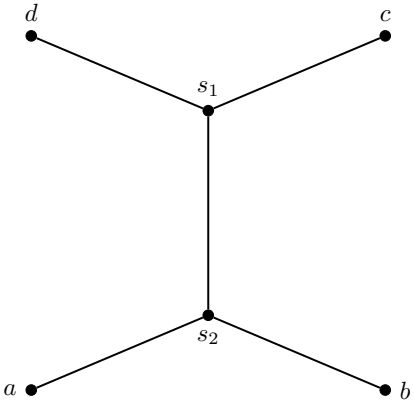
Four vertices – Fermat points





- s_1: Fermat point for $\triangle adc$
- s_2: Fermat point for $\triangle abs_1$
- Length of network: ≈ 27.852
- These are not Steiner points, since the incident edges do not form $120°$ angles.

Four vertices – Steiner tree



- Total length 27.321
- Network with Fermat points: 27.852
- MST: 30

Steiner Network Method

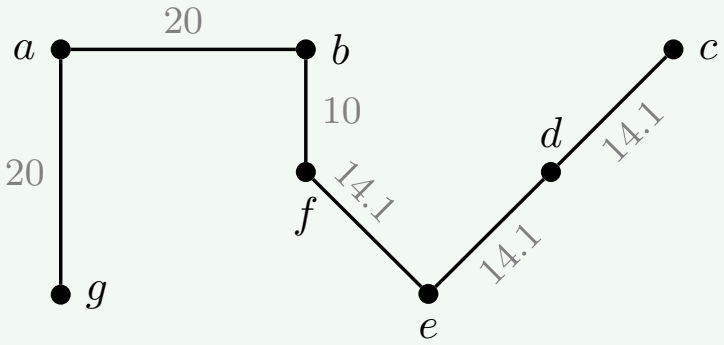
- We will not attempt to find a minimum Steiner tree.
- Instead, we use the ideas behind Steiner trees to find a network shorter than a minimum spanning tree, if possible.

Steiner Network Method

- Step 1. Find the MST of the network
- Step 2. Form a triangle using two existing edges of the minimum spanning tree. If all angles of this triangle are less than 120°, find its Fermat point.
- Step 3. Update the network by removing the two edges from the MST used in Step 2 and adding edges from the corresponding vertices to the Fermat point.
- Step 4. Repeat steps 2 and 3 until all possible triangles have been considered

Steiner Network Method – example

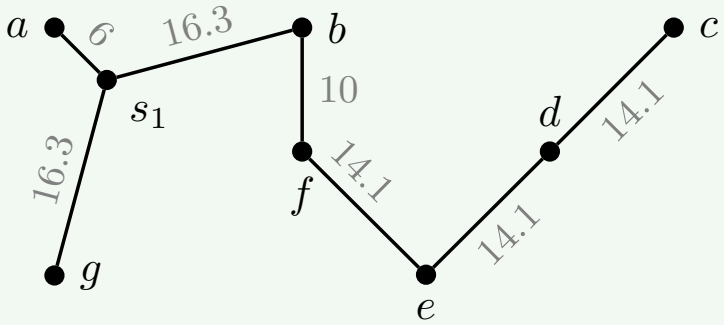
Example



- We begin with vertices a, b, g, since the angle at a is $90°$.

Steiner Network Method – example

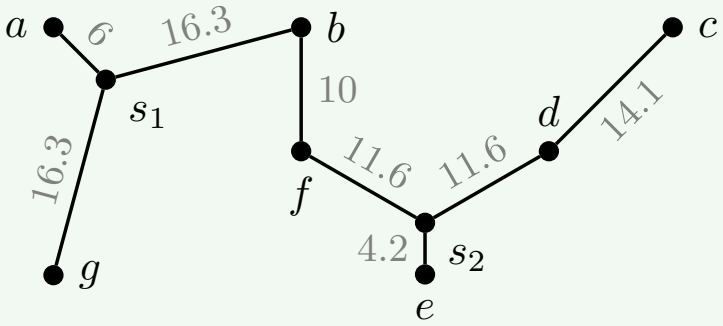
Example



- We begin with vertices a, b, g. Using Torricelli's Construction, we obtain the point s_1.

Steiner Network Method – example

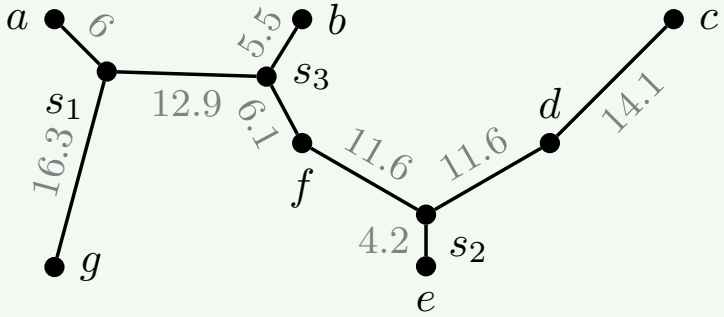
Example



- Next, consider vertices f, e, d. Since the angle at e is $90°$, Torricelli's Construction gives the point s_2.
- At this stage, the angle conditions leave only one triangle to consider: b, f, s_1.

Steiner Network Method – example

Example



- Finally, using vertices b, f, s_1, we obtain the point s_3.
- Total length: 88.3
- Original (MST) length: 92.3

Remark

- Although we have not fully solved the optimization problem for networks containing more than three points,
- the Steiner Network Method provides a quick and simple way to identify possible improvements to a minimum spanning tree.

Trees and networks

- Shortest networks
- Steiner Trees
- Metric Traveling Salesman Problem
- Flow and capacity
- Rooted trees

Traveling Salesman Problem

- How should a delivery service plan its route through a city to ensure each customer is reached?
- A traveling salesman has customers in numerous cities. He must visit each of them and return home, but wishes to do this with the least total cost
- The Traveling Salesman Problem asks for a shortest cycle that visits every city exactly once and returns to the starting city; in graph terms, this is a shortest Hamiltonian cycle in a weighted complete graph.

Metric Traveling Salesman Problem

- Abbreviated as mTSP
- Only considers the scenarios where the weights satisfy the *triangle inequality*
- For a weighted complete graph $K = (V, E, \omega)$, given any three vertices x, y, z

$$\omega(xy) + \omega(yz) \geq \omega(xz)$$

mTSP Algorithm

- mTSP Algorithm combines three ideas we have studied so far – Eulerian circuits, Hamiltonian cycles and MST
- We duplicate every edge of a minimum spanning tree, making all vertex degrees even and thus producing a graph with an Eulerian circuit.
- We then convert this Eulerian circuit into a Hamiltonian cycle by skipping repeated vertices.
- This procedure is guaranteed to produce a Hamiltonian cycle only when the underlying graph is complete.
- It may still find a proper Hamiltonian cycle when the graph is not complete, but cannot be guaranteed to do so.

mTSP Algorithm

- Input: weighted complete graph K_n, where the weight function ω satisfies the triangle inequality
- Steps:
  1. Find a MST T for K_n
  2. Duplicate all the edges of T to obtain T^*
  3. Find an Eulerian circuit for T^*
  4. Convert the Eulerian circuit into a Hamiltonian cycle by skipping any previously visited vertex (except for the starting and ending vertex)
  5. Calculate the total weight
- Output: Hamiltonian cycle for K_n

mTSP Algorithm – example

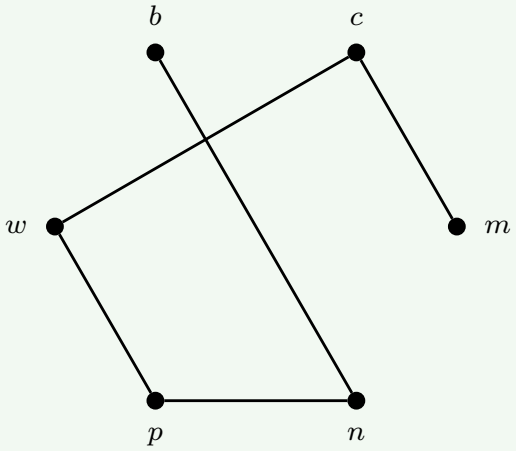
Example

Alice must visit six cities and needs to minimize the total distance

	Boston	Charlotte	Memphis	New York	Philadelphia	D.C.
Boston	.	840	1316	216	310	440
Charlotte	840	.	619	628	540	400
Memphis	1316	619	.	1096	1016	876
New York City	216	628	1096	.	97	228
Philadelphia	310	540	1016	97	.	140
Washington, D.C.	440	400	876	228	140	.

mTSP Algorithm – example

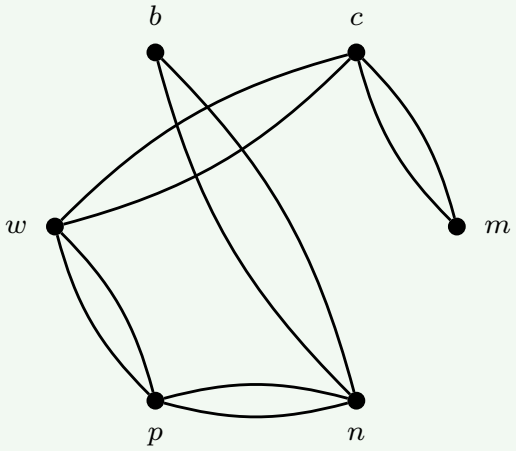
Example



- Step 1. MST

mTSP Algorithm – example

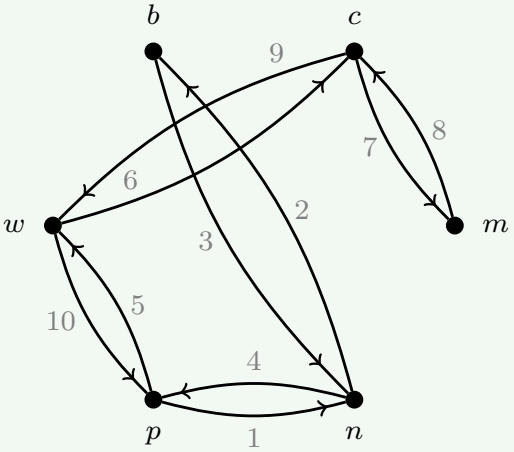
Example



- Step 2. Duplicate all edges of the MST

mTSP Algorithm – example

Example



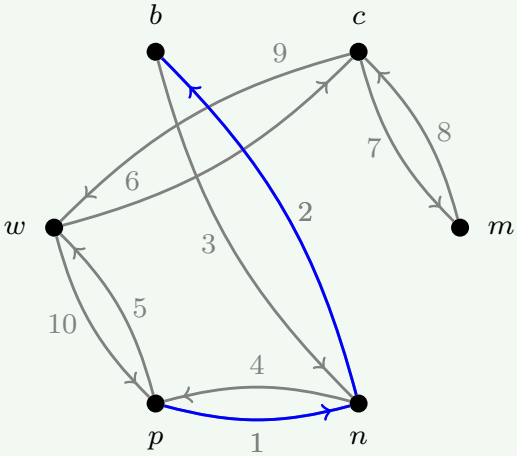
- Step 3. An Eulerian circuit is $pnbnpwcmcwp$

mTSP Algorithm – example

Example

Step 4

- Follow the Eulerian circuit until we reach b.
- Since we want a Hamiltonian cycle, we must skip any vertex that has already been visited, so we do not return to n.
- The next vertex along the circuit that has not been previously visited is w

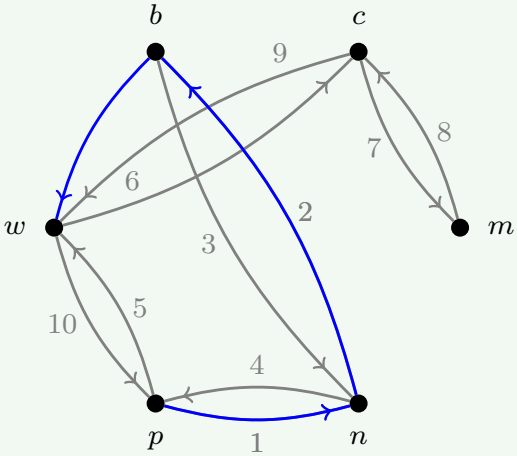


mTSP Algorithm – example

Example

Step 4

- Add the edge from b to w

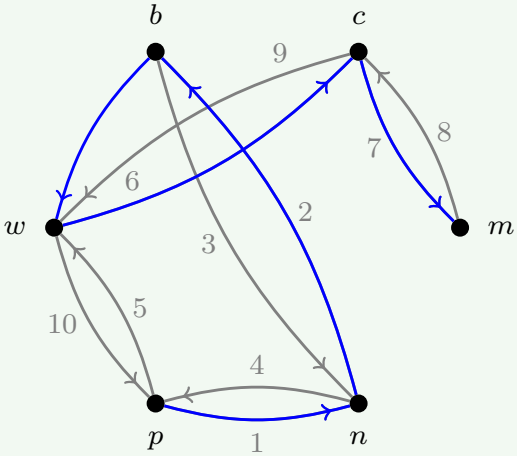


mTSP Algorithm – example

Example

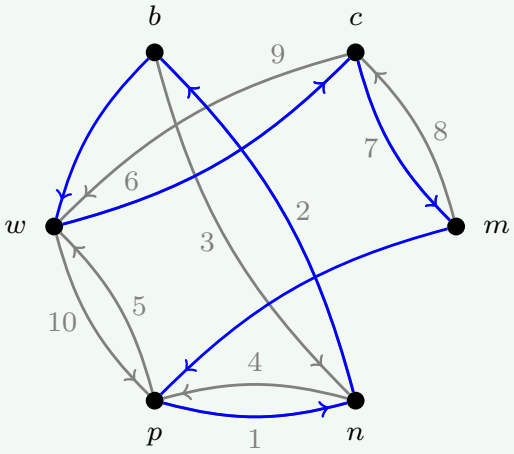
Step 4

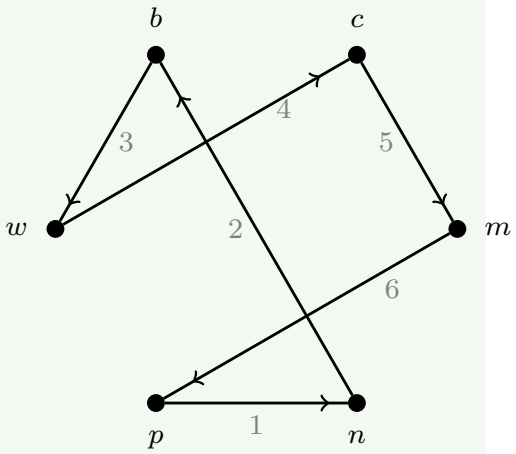
- Continue following the Eulerian circuit until m is reached.
- Again, we skip c because it has already been visited, and at this stage the only remaining move is to return to p.



mTSP Algorithm – example

Example





mTSP Algorithm – example

Example

- Total weight of the MST: 1472 – the worst possible Hamiltonian cycle that can arise from it will have weight at most two times that 2944, due to the doubling of the edges
- Total weight of the Hamiltonian cycle: 2788
- The actual optimal Hamiltonian cycle has total weight 2781.
- Thus, the relative error of our Hamiltonian cycle is approximately 0.25%.

Trees and networks

- Shortest networks
- Steiner Trees
- Metric Traveling Salesman Problem
- Flow and capacity
- Rooted trees

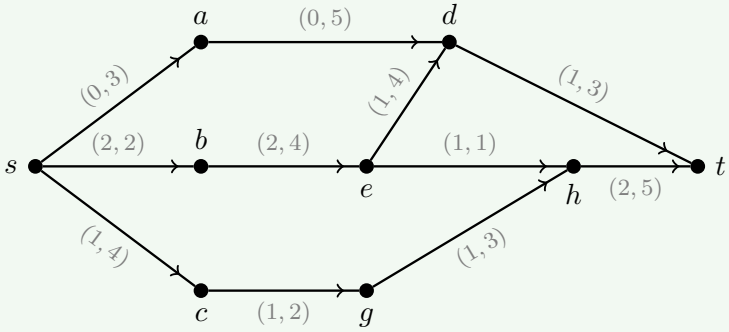
A different notion of a network

Definition

- A *network* is a connected digraph where each arc e has an associated nonnegative integer $c(e)$, called a *capacity*.
- The network has a designated starting vertex s, called the *source*, and a designated ending vertex t, called the *sink*.
- A *flow* f is a function that assigns a value $f(e)$ to each arc e of the network

Network – example

Example



- Each arc is given a two-part label, $(f(e), c(e))$
- The first component is the flow along the arc
- The second component is the capacity

Network

- The terms source and sink are motivated by the analogy of water flowing through a system of pipes, from the source to the sink.
- In this analogy, flow travels in the direction of the arcs, no arc can carry more than its capacity, and for every intermediate vertex the total inflow equals the total outflow.

Feasible flow

Definition

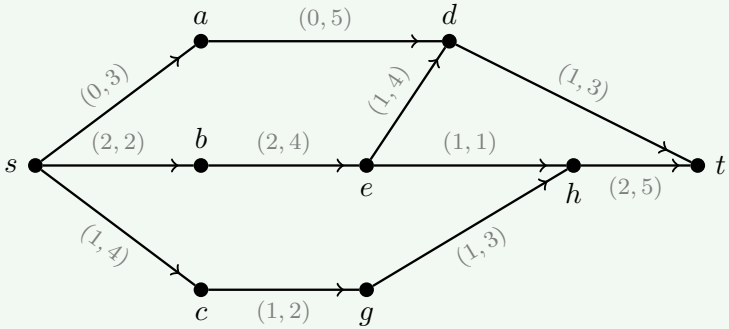
For a vertex v, let $f^-(v)$ represent the total flow entering v and $f^+(v)$ represent the total flow exiting v. A flow is *feasible* if it satisfies the following conditions

- $f(e) \geq 0$ for all arcs e
- $c(e) \geq f(e)$ for all arcs e
- $f^+(v) = f^-(v)$ for all vertices other than s and t
- $f^-(s) = f^+(t) = 0$

- The notation for inflow and outflow parallels that for in-degree and out-degree, except that here we sum the flow values on the arcs entering or leaving a vertex.
- The requirement that a flow is non-negative indicates the flow must travel in the direction of the arc, as a negative flow would indicate items going in the reverse direction
- The final condition is not necessary in theory, but more logical in practice and simplifies our analysis of flow problems

Network – example

Example



- $f^+(s) = 0 + 2 + 1 = 3,\ f^-(t) = 1 + 2 = 3$
- $c(ad) = 5,\ f(ad) = 0,\ c(ad) > f(ad)$
- $f^+(b) = f^-(b) = 2$

Maximum flow

Definition

- The *value* of a flow is defined as

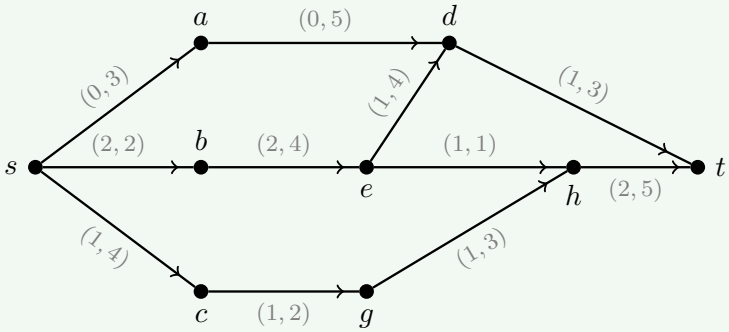
$$|f| = f^+(s) = f^-(t),$$

  i.e. the amount exiting the source which must also equal the flow entering the sink
- A *maximum flow* is a feasible flow of largest value

- In practice, we use integer values for the capacity and flow, though this is not required.
- How can we find a maximum flow?

Network – example

Example



- Feasible flow, value 3
- Finding maximum flow is not as simple as putting every arc at capacity – e.g. if we had a flow of 5 along arc ad, flow along dt would be 6, more than its capacity

Slack

Definition

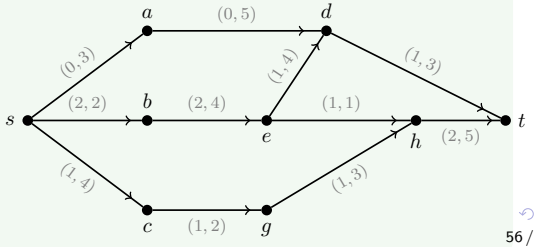
Let f be a flow along a network. The *slack* k of an arc is the difference between its capacity and flow

$$k(e) = c(e) - f(e).$$

Slack edge: $k(e) > 0$

Example

- $k(sa) = 3$, $k(sc) = 3$, $k(sb) = 0$
- We may want to increase flow along sa and sc but not sb

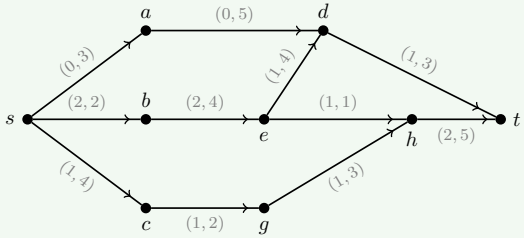


Chain

Definition

A *chain* K is a path in a digraph where the direction of the arcs are ignored

Example



- Both $sadt$ and $sadeht$ are chains
- $sadeht$ is not a directed path

Augmenting Flow Algorithm (Ford-Fulkerson Algorithm)

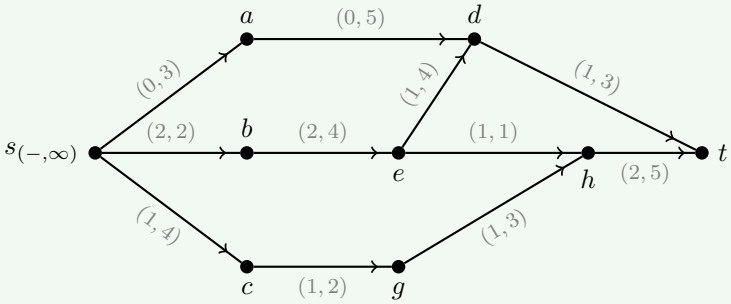
- This algorithm is similar in spirit to Dijkstra's Algorithm, which finds shortest paths in a graph or digraph.
- Vertices are assigned two-part labels that help us construct a chain along which the flow can be increased.
- The label consists of two parts, the second component is denoted by $\sigma(v)$
- Input: Network $G = (V, E, c)$, where each arc is given a capacity c, and a designated source s and sink t
- Output: Maximum flow f

Augmenting Flow Algorithm – steps

1. Label s with $(-, \infty)$, set $\sigma(v) = \infty$ for other vertices
2. Choose a labeled vertex x, scan x:
   a. For any arc yx, if $f(yx) > 0$ and y is unlabeled, then label y with $(x^-, \sigma(y))$, where $\sigma(y) = \min\{\sigma(x), f(yx)\}$
   b. For any arc xy, if $k(xy) > 0$ and y is unlabeled, then label y with $(x^+, \sigma(y))$, where $\sigma(y) = \min\{\sigma(x), k(xy)\}$
3. If t has been labeled, go to Step 4. Otherwise, choose another labeled vertex that has not yet been scanned and return to Step 2. If all labeled vertices have been scanned and t is still unlabeled, then f is a maximum flow.
4. Find an s-t chain K by backtracking from t to s using the vertex labels. Along the edges of K, increase the flow by $\sigma(t)$ units if they are in the forward direction and decrease by $\sigma(t)$ units in they are in the backward direction. Remove all vertex labels except that of s and return to Step 2

Augmenting Flow Algorithm – example

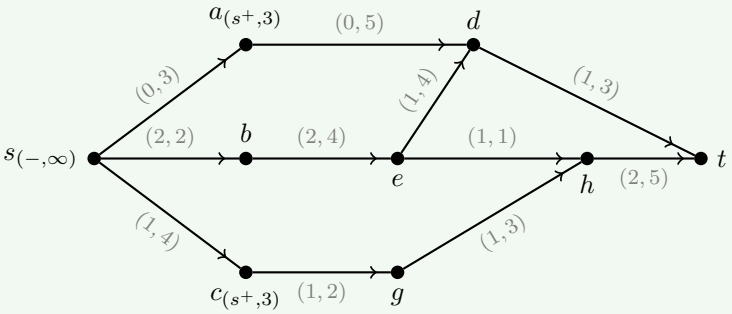
Example



- Step 1. Label s as $(-,\infty)$

Augmenting Flow Algorithm – example

Example

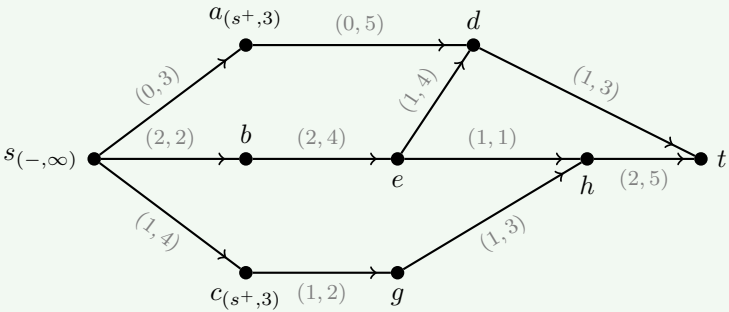


Step 2.

- There are no arcs to s.
- Arcs out of s: sa, sb, sc, with slack $3, 0, 3$
- Label a with $(s^+, 3)$, c with $(s^+, 3)$. Leave b unlabeled since sb has slack 0

Augmenting Flow Algorithm – example

Example

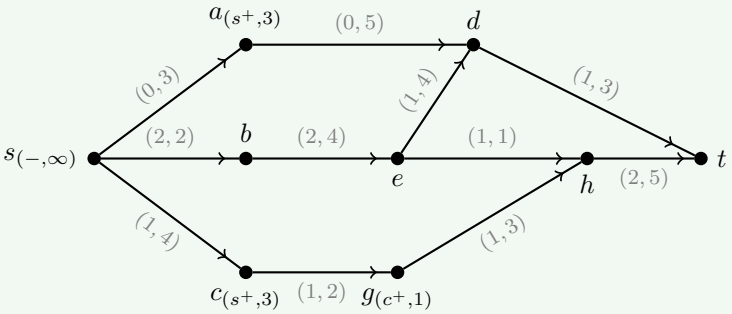


Step 3.

- Since t is not labeled, choose another labeled vertex that has not yet been scanned —either a or c.
- Choose c.

Augmenting Flow Algorithm – example

Example

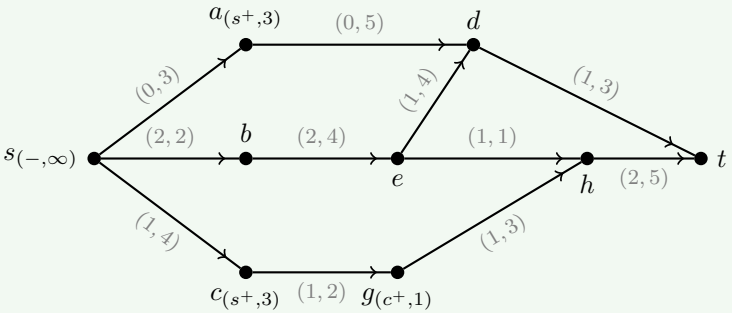


Step 2.

- The only arc entering c comes from a labeled vertex.
- Consider the arcs out of c – there is only cg with slack of 1
- Label g as $(c^+, 1)$, where $\sigma(g) = \min\{\sigma(c), k(cg)\} = \min\{3, 1\} = 1$

Augmenting Flow Algorithm – example

Example

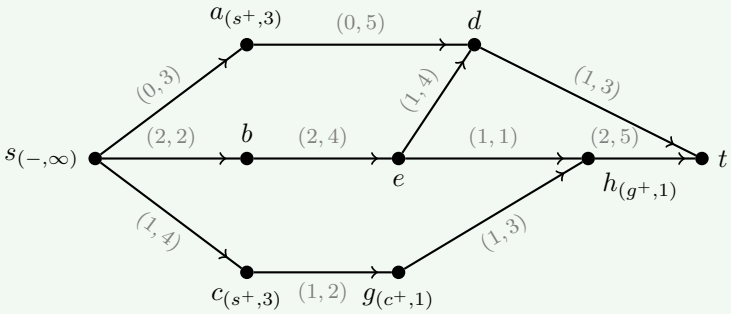


Step 3.

- t is not labeled
- We can scan either a or g
- Choose g

Augmenting Flow Algorithm – example

Example

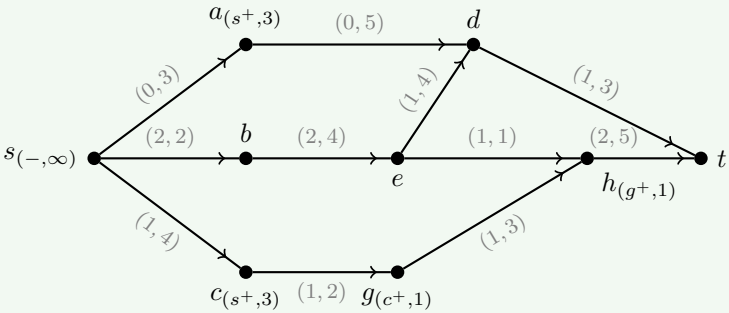


Step 2.

- The only arc entering g comes from a labeled vertex.
- The only arc out of g is gh
- Label h as $(g^+, 1)$, where $\sigma(h) = \min\{\sigma(g), k(gh)\} = \min\{1, 2\} = 1$

Augmenting Flow Algorithm – example

Example

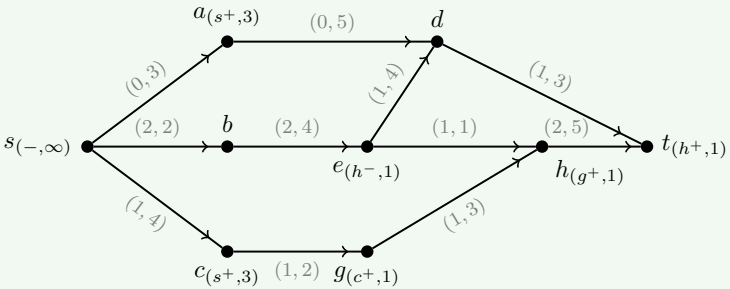


Step 3.

- t is not labeled, we can choose a or h
- Choose h

Augmenting Flow Algorithm – example

Example

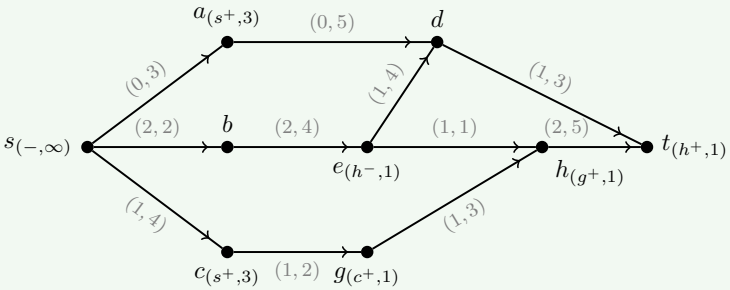


Step 2.

- There is one unlabeled vertex with an arc into h, namely e. Label e with $(h^-, 1)$, since $\sigma(e) = \min\{\sigma(h), f(eh)\} = \min\{1, 1\} = 1$.
- Label t as $(h^+, 1)$, where $\sigma(t) = \min\{\sigma(h), k(ht)\} = \min\{1, 3\} = 1$

Augmenting Flow Algorithm – example

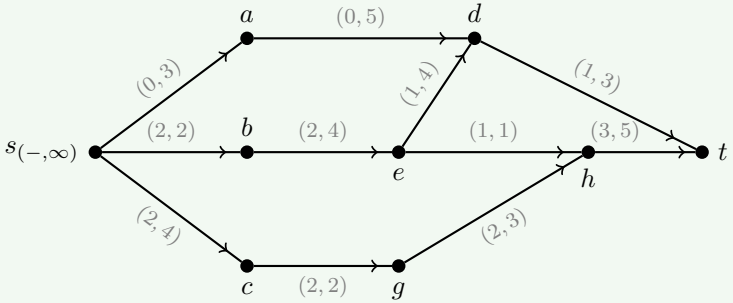
Example



- Step 3. t is labeled, go to Step 4
- Step 4. Backtracking from t to s using slack edges gives the chain $scght$.

Augmenting Flow Algorithm – example

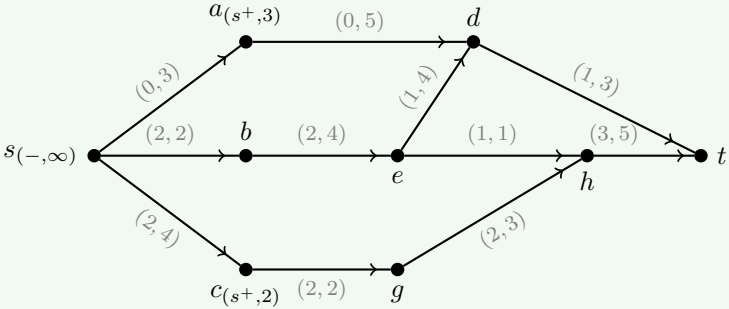
Example



- Step 4. Backtracking from t to s using slack edges gives the chain $scght$
  - Increase the flow by $\sigma(t) = 1$ units along each of these edges since all are in the forward direction.
  - Update the network flow and remove all labels except for s

Augmenting Flow Algorithm – example

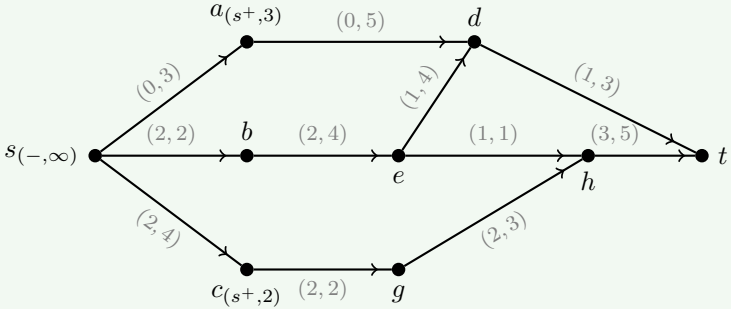
Example



- Step 2. Label a with $(s^+, 3)$ and c with $(s^+, 2)$
- Step 3. Choose c to scan

Augmenting Flow Algorithm – example

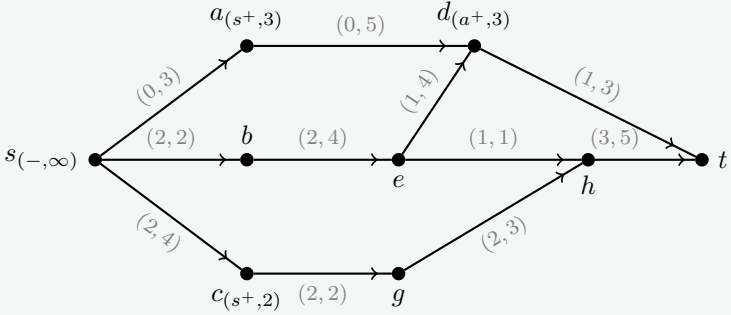
Example



- Step 2. No vertex to label, $k(cg) = 0$
- Step 3. Choose a to scan

Augmenting Flow Algorithm – example

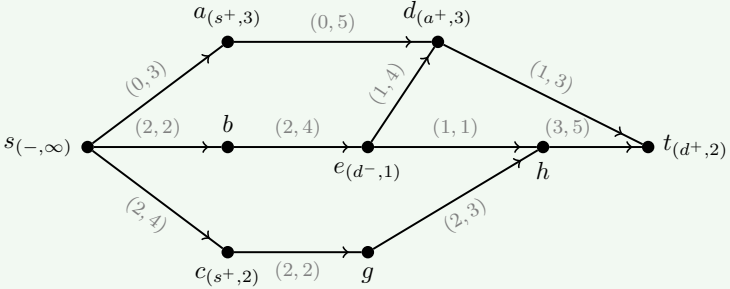
Example



- Step 2. Label d with $(a^+, 3)$, where $\sigma(d) = \min\{\sigma(a), k(ad)\} = \min\{3, 5\} = 3$
- Step 3. Only unscanned labeled vertex is d

Augmenting Flow Algorithm – example

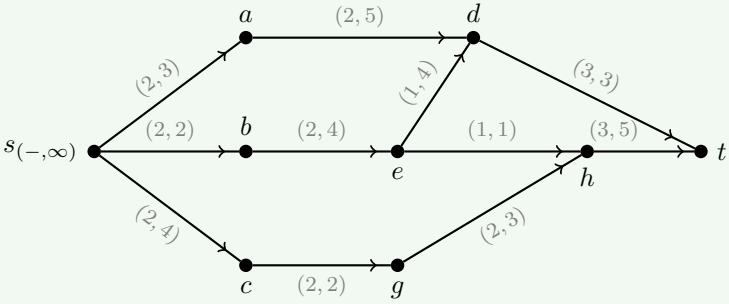
Example



- Step 2. Label e with $(d^-, 1)$, where $\sigma(e) = \min\{\sigma(d), f(ed)\} = \min\{3, 1\} = 1$. Label t with $(d^+, 2)$, $\sigma(t) = \min\{\sigma(d), k(dt)\} = \min\{3, 2\} = 2$
- Step 3. t is labeled, go to step 4
- Step 4. Backtracking gives the chain $sadt$

Augmenting Flow Algorithm – example

Example

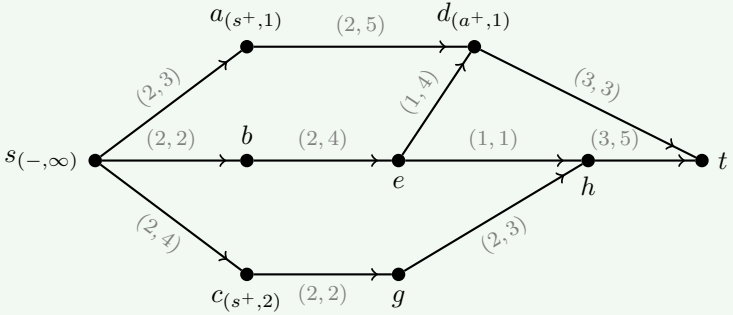


Step 4.

- Find chain $sadt$, increase the flow by $\sigma(t) = 2$ units along each of these edges since all are in the forward direction
- Update the network flow and remove all labels except for s

Augmenting Flow Algorithm – example

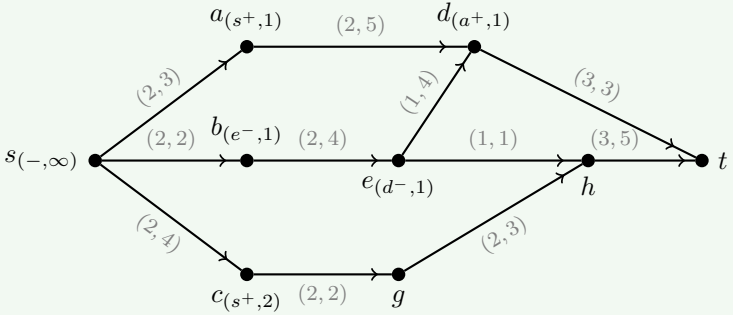
Example



- Step 2. Label a and c
- Step 3. Choose a
- Step 2. Label d
- Step 3. Choose d

Augmenting Flow Algorithm – example

Example



- Step 2. Label e as $(d^-, 1)$. Vertex t is not labeled because $k(dt) = 0$
- Step 3. Choose e
- Step 2. Label b
- Step 3. No vertices to be labeled, we get a maximum flow

Augmenting Flow Algorithm

- When the Augmenting Flow Algorithm halts, a maximum flow has been achieved, though understanding why this flow is indeed maximum requires additional terminology and results

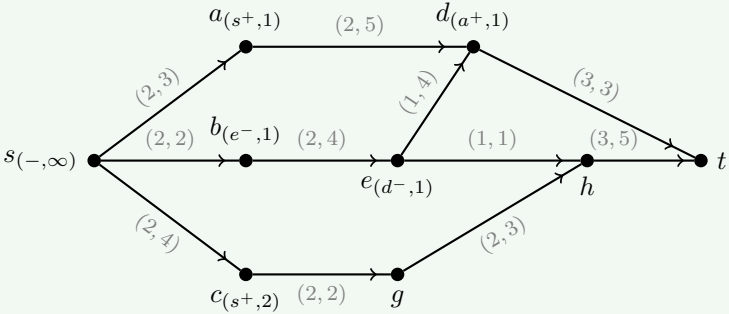
Cut

Definition

- Let P be a set of vertices and $\overline{P} = V - P$.
- A *cut* $(P, \overline{P})$ is the set of all arcs xy where $x \in P$ and $y \in \overline{P}$
- An $s - t$ *cut* is a cut in which $s \in P$, $t \in \overline{P}$

Cut – example

Example



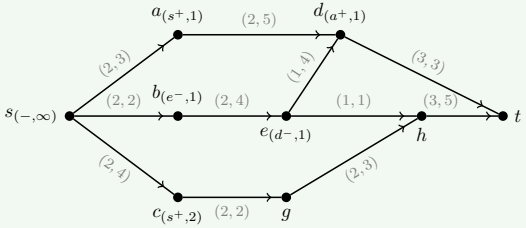
- $P = \{s, a, e, g\}$, $\overline{P} = \{b, c, d, h, t\}$
- $(P, \overline{P}) = \{sb, sc, ad, ed, eh, gh\}$
- Note: $be \notin (P, \overline{P})$ because $b \in \overline{P}$, $e \in P$

Capacity

Definition

The *capacity* of a cut, $c(P, \overline{P})$, is defined as the sum of the capacities of the arcs that comprise the cut.

Example



- $P = \{s, a, e, g\}$, $\overline{P} = \{b, c, d, h, t\}$, $(P, \overline{P}) = \{sb, sc, ad, ed, eh, gh\}$, $c(P, \overline{P}) = 2 + 4 + 5 + 4 + 1 + 3 = 19$
- $P = \{s\}$, $c(P, \overline{P}) = 9$

Max Flow-Min Cut

Theorem

In any directed network, the value of a maximum $s-t$ flow equals the capacity of a minimum $s-t$ cut.

- The difficulty in using this theorem directly is that one must still identify a minimum cut.
- We can use the vertex labeling procedure to obtain our minimum cut.

Min-Cut Method

Steps

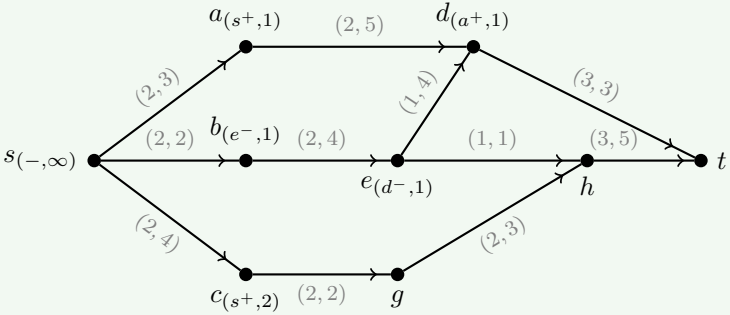
1. Let $G = (V, A, c)$ be a network with a designated source s and sink t and each arc is given a capacity c
2. Apply the Augmenting Flow Algorithm
3. Define an $s - t$ cut $(P, \overline{P})$ where P is the set of labeled vertices from the final implementation of the algorithm
4. $(P, \overline{P})$ is a minimum $s - t$ cut for G

Note

In practice, we can perform the Augmenting Flow Algorithm and the Min-Cut Method simultaneously, thus finding a maximum flow and providing a proof that it is maximum (through the use of a minimum cut) in one complete procedure.

Min-Cut Method – example

Example



- $P = \{s, a, b, c, d, e\}$, $\overline{P} = \{g, h, t\}$
- $(P, \overline{P}) = \{dt, eh, cg\}$
- $c(P, \overline{P}) = 3 + 1 + 2 = 6$

Trees and networks

- Shortest networks
- Steiner Trees
- Metric Traveling Salesman Problem
- Flow and capacity
- Rooted trees

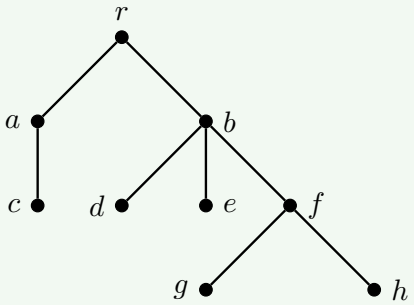
Definition

Definition

- A *rooted tree* is a tree T with a special designated vertex r, called the *root*
- The *level* of any vertex in T is defined as the length of its shortest path to r
- The *height* of a rooted tree is the largest level for any vertex in T

Example

- root r: level 0
- a, b: level 1
- c, d, e, f: level 2
- g, h: level 3
- height of the tree: 3



Terminologies

Definition

Let T be a tree with root r. Then for any vertices x and y

- y is on the unique path from x to r: x is a *descendant* of y; y is an *ancestor* of x
- x is a descendant of y and exactly one level below y: x is a *child* of y, y is a *parent* of x
- x is a *sibling* of y if x and y have the same parent.

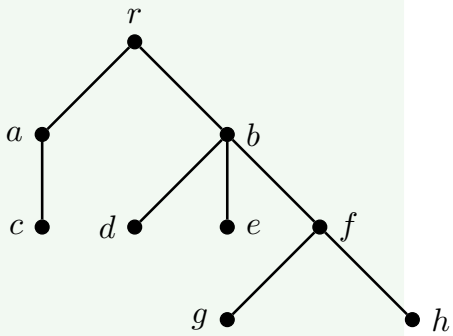
Note

Analogy: family tree

Terminologies – example

Example

- parent of a: r
- child of a: c
- parent of e: b
- e has no children
- ancestors of g: f, b, r – unique path from g to r is $gfbr$
- descendants of b: d, e, f, g, h
- siblings of d: e, f



Depth-First Search Tree

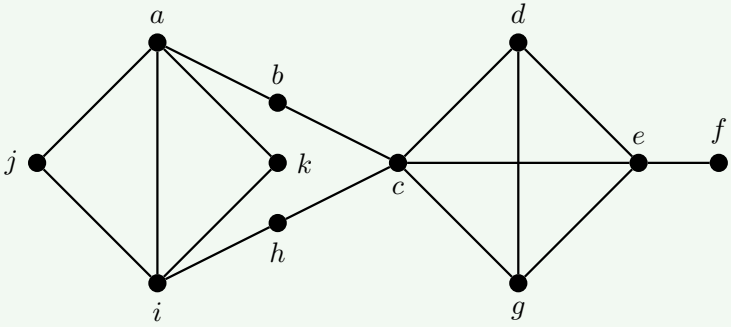
- The main idea is to follow a path as far as possible from the root before backtracking.
- If this path does not include all vertices, we backtrack and build further branches to obtain a tree.
- The formal description of the algorithm uses an ordered list of neighbors for each vertex, and this order determines which new vertex is added next.
- For simplicity, we will always use an alphabetical order when considering neighbor lists
- Input: Simple (no multi-edges or loops) connected graph $G = (V, E)$ and a designated root vertex r
- Output: Depth-first search tree T

Depth-First Search Tree – steps

1. Initialize the DFS tree $T = (V', E')$ with $V' = \{r\}$ and $E' = \emptyset$. Set r as the current vertex v.
2. Select the first unvisited neighbor x of the current vertex v. Add vertex x and edge vx to T, and recursively repeat Step 2 with x as the new current vertex until no unvisited neighbors remain.
3. If all vertices of G are now in T, then T is the depth-first search tree. Otherwise, backtrack along the path from the last visited vertex to the root until you find a vertex v with an unvisited neighbor. Use v as the current vertex and repeat Step 2.

Depth-First Search Tree – example

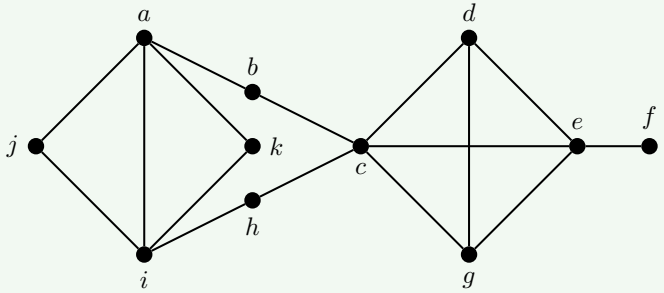
Example

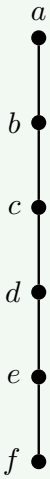


- Suppose a is the root
- Step 1. Current vertex is a
- Step 2. Add b, c, d, e, f in order. This process stops at f, since f has no unvisited neighbors in G.

Depth-First Search Tree – example

Example

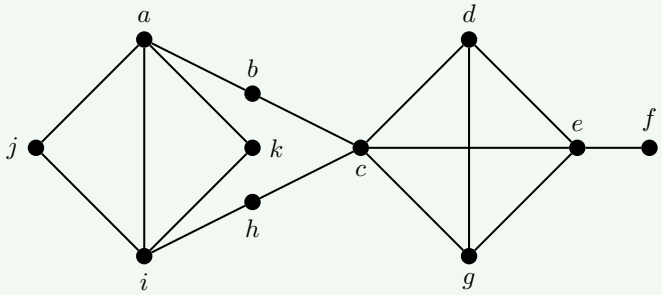


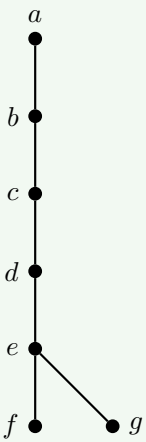


- Step 3. Backtracking along the path, the first vertex with an unvisited neighbor is e
- Step 2. Add edge eg and vertex g to T

Depth-First Search Tree – example

Example

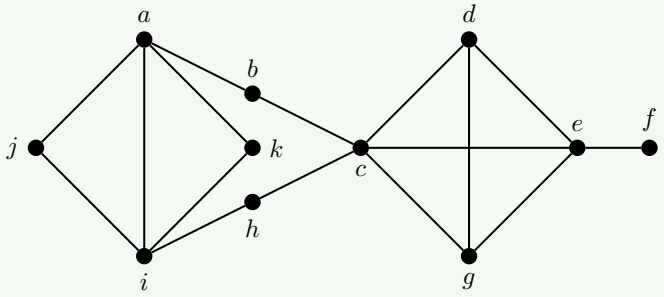


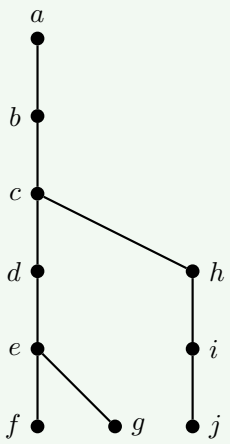


- Step 3. Backtracking along the path from g to a, the first vertex with an unvisited neighbor is c
- Step 2. Add h, i, j in order.

Depth-First Search Tree – example

Example

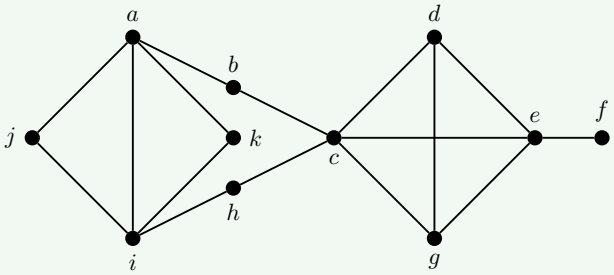


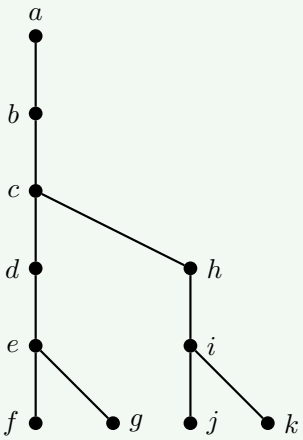


- Step 3. Backtracking along the path from j to a, the first vertex with an unvisited neighbor is i
- Step 2. Add k

Depth-First Search Tree – example

Example





- T contains all the vertices of G
- The resulting tree is the depth-first search tree
- The tree has height 5, with one vertex at each of levels 1 and 2, two vertices at each of levels 3 and 4, and four vertices at level 5.

Remark

- If the graph is not connected, we can slightly adjust the algorithm and get a forest as the output
- Search for unvisited vertices when no vertex with unvisited neighbors can be found

Breadth-First Search Tree

- Main objective is to add as many neighbors of the root as possible in the first step
- At each subsequent step, we add all unvisited neighbors of the most recently added vertices.
- As with depth-first search, we will use alphabetical ordering of neighbor lists.
- Input: Simple connected graph $G = (V, E)$ and a designated root vertex r
- Output: Breadth-first tree T

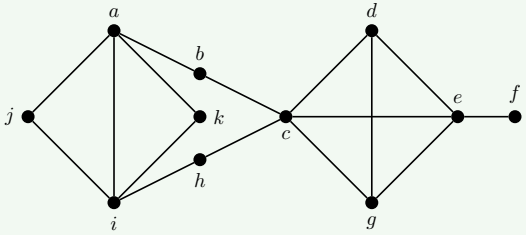
Breadth-First Search Tree – steps

1. Initialize the BFS tree $T = (V', E')$ with the root vertex r, i.e., $V' = \{r\}$, and mark r as visited.
2. Add all neighbors of r to V', and add the corresponding edges from r to each neighbor to E'. Mark all these neighbors as visited. Let this set of newly added vertices be the current level.
3. For each vertex v in the current level (in alphabetical order):
   - Add all unvisited neighbors x of v to V'.
   - Add the edge (v, x) to E'.
   - Mark each such neighbor x as visited.

   Let the collection of all such newly added vertices form the next level.
4. If T now includes all vertices of G, the process is complete. Otherwise, repeat step 3 using the next level as the current level.

Breadth-First Search Tree – example

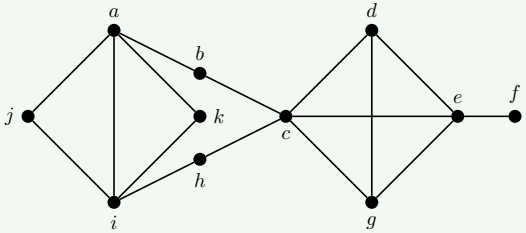
Example



- Let's consider the same example
- Take a as the root
- Step 1. Add a, b, i, j, k
- Step 2. Current level: b, i, j, k

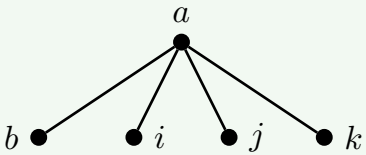
Breadth-First Search Tree – example

Example



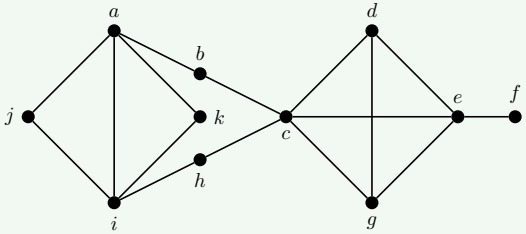
Step 3

- Vertex b: add neighbor c, and edge bc
- Vertex i: add neighbor h and edge ih
- j and k do not have any unvisited neighbors

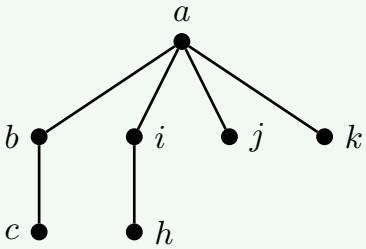


Breadth-First Search Tree – example

Example

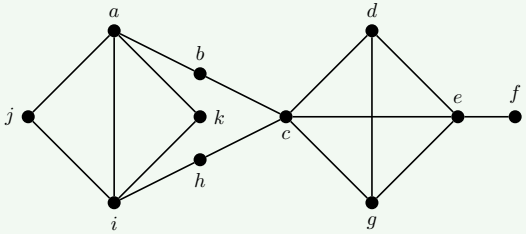


- Step 4. T does not contain all vertices, repeat step 3. Current level: c, h
- Step 3.
  - Vertex c: unvisited neighbors d, e, g
  - Vertex h has no unvisited neighbors

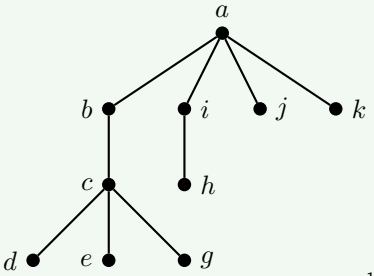


Breadth-First Search Tree – example

Example

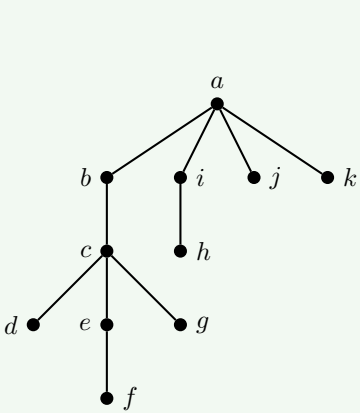


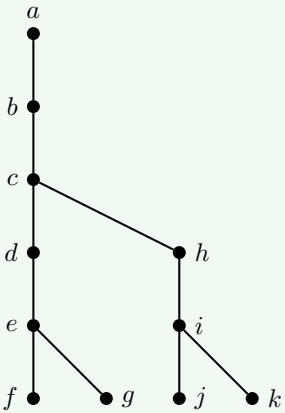
- Step 4. T does not contain all vertices, repeat step 3. Current level: d, e, g
- Step 3.
  - Vertex d has no unvisited neighbors
  - Vertex e: add neighbor f and edge ef
  - Now we have all vertices



Breadth-First Search Tree – example

Example

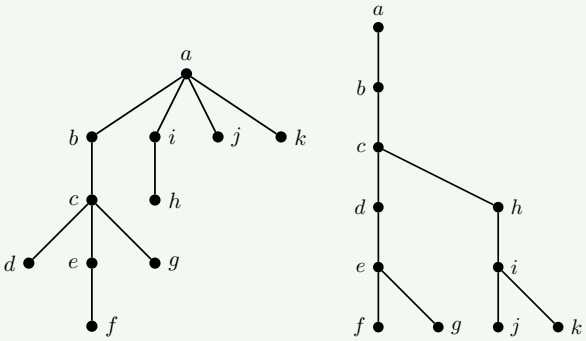




- Left: BFS tree; Right: DFS tree
- BFS trees are often shorter in height than their DFS counterparts.

Breadth-First Search Tree – example

Example



- The BFS tree has height 4, with four vertices on level 1, two on level 2, three on level 3, and one on level 4.
- The DFS tree has height 5, with one vertex at each of levels 1 and 2, two vertices at each of levels 3 and 4, and four vertices at level 5.